

run-api-app-challenge-matrix.sh

§6.AE / §6.X / §6.AG gate entry point for the :api-app module's Compose UI Challenge tests (Challenge01..Challenge04 under lava.api.app.challenges).

Script: scripts/run-api-app-challenge-matrix.sh

What it does

The :api-app sibling of scripts/run-challenge-matrix.sh. THIN GLUE: it delegates emulator boot + lifecycle to the Containers submodule's cmd/emulator-matrix CLI (per §6.X / §6.AG — the device MUST come from a Containers-orchestrated cold-booted emulator AVD, NEVER a live/physical ADB device). It forwards --runner=auto (Linux→containerized via /dev/kvm, macOS→host-direct+HVF), the :api-app debug APK, the Challenge test class, the AVD matrix, and the evidence dir; the CLI's exit code is the gate's source of truth.

Usage

```
./scripts/run-api-app-challenge-matrix.sh [--test-class
lava.api.app.challenges.ChallengeNN] \
    [--avds "name:api:form,..."] [--evidence-dir <dir>] [--no-
build] \
    [--boot-timeout <duration>] [--test-timeout <duration>] \
    [--container-image <ref>] [--container-runtime <podman|
docker>] \
    [--build-type debug|releaseTest]
```

--build-type — release-variant device testing (2026-09-22)

Default debug — fully backward compatible, omitting this flag behaves exactly as before it existed. --build-type releaseTest targets a new Gradle build type on api-app/build.gradle.kts that mirrors release (same proguard-rules.pro/postprocessing shape actually shipped) but

sets `isDebuggable=true` so instrumentation can attach at all — Android instrumentation cannot attach to a non-debuggable APK on a non-rooted device, a hard platform constraint, so `release` itself can never gain a `connectedReleaseAndroidTest` task. This script forwards `-PlavaTestReleaseVariant=true` to the local `./gradlew assemble` step and both `--build-type releaseTest` and `--gradle-property lavaTestReleaseVariant=true` to the Containers emulator-matrix CLI, whose own internal `./gradlew` invocation needs that property to select the same `testBuildType`.

```
./scripts/run-api-app-challenge-matrix.sh --no-build --build-type
    releaseTest \
    --test-class
        lava.api.app.challenges.Challenge05ApiEmbedSourceHashMatchesTest
```

--test-timeout pass-through — LVA-161 (2026-08-26)

emulator-matrix defaults `--test-timeout` to **10 minutes** (`cmd/emulator-matrix/main.go:123`), and that budget covers the **TEST step only** — cold boot is timed separately as `boot_seconds`. This script previously never set it, so every run inherited that default.

The default is far below a real whole-module sweep: a Challenge run is a **single** gradle invocation covering every selected class, and gradle writes its JUnit XML at the **end** of that invocation. A kill at the timeout therefore destroys the results of **every** class, including those that had already passed — the matrix then reports the entire selection as failed when nothing about the product was wrong. The sibling `scripts/run-challenge-matrix.sh` recorded the measured case: a run killed at `test_seconds=600.02` with signal: killed after completing 81 of 104 tests.

This script now accepts `--test-timeout <duration>` and forwards it verbatim to the CLI when set; when unset, the flag is omitted entirely and the CLI default applies. This script performs **no** sizing of its own — the caller owns that. The pipeline's Challenge phase (`scripts/pipeline/phase-02-test-challenge.sh`) derives the value as `300s + 45s × <selected class count>` and passes it in; an operator invoking this script directly for more than a handful of classes should pass a sized value rather than accept 10m.

A run killed at the timeout is **not** a test failure: `test_seconds` at the budget plus `signal: killed` plus an absent JUnit XML is the signature. The honest remedy is to raise `--test-timeout` or narrow the class selection and re-run for a real verdict.

`--boot-timeout` is the separate, pre-existing knob for cold-boot headroom on a loaded host (contention, not a product defect); the two budgets do not interact.

Constitutional bindings

§6.AE (per-app Challenge matrix), §6.X (container-submodule emulator wiring), §6.AG (Containers-driven, no live device), §6.AH (VM/container, never host-direct), §6.I (per-AVD attestation).